

Cryptography and Network Security

Integrated Situation - Technical Report

Gad Rubimbura
Registration Number: 2200012345

September 24, 2026

Contents

1. Introduction
2. Risk Assessment
3. Encryption and Integrity Application
4. Network Traffic Filtering
5. Conclusion

1 Introduction

A polytechnic institute stores student records on a central server and transfers files between two campuses[cite: 1, 5, 6]. A security review found weak staff passwords, outdated software, unencrypted file transfers and guest-network access to the records server, and repeated connection attempts from an unfamiliar external address were observed[cite: 1, 5, 6]. This report documents the risk assessment, the Python encryption and integrity toolkit, and the firewall rules produced to address these findings[cite: 1, 5, 6]. All source files are in the GitHub repository (Section 5).

2 Risk Assessment

2.1 Assets, vulnerabilities and consequences

- **R1 (Staff accounts credentials):** Weak passwords; repeated external login attempts → Account takeover; records read, altered, or deleted; misuse under a staff identity[cite: 5, 6].
- **R2 (Student records server):** Guest network can reach the server; outdated software → Scanning and exploitation of known flaws; data breach, ransomware, outage; reputational and legal harm[cite: 5, 6].
- **R3 (Record files in transit):** Unencrypted file transfer → Eavesdropping or man-in-the-middle; records disclosed or silently modified[cite: 5, 6].

2.2 Risk ranking

Likelihood and impact are scored from 1 (low) to 5 (high); the score is their product[cite: 5, 6].

- **Rank 1: R1 (Weak staff passwords)** — Likelihood: 5, Impact: 5, Score: 25[cite: 5, 6].
- **Rank 2: R2 (Guest access / outdated software)** — Likelihood: 4, Impact: 5, Score: 20[cite: 5, 6].
- **Rank 3: R3 (Unencrypted transfers)** — Likelihood: 3, Impact: 4, Score: 12[cite: 5, 6].

2.3 Recommended controls

- **For R1:** Strong password policy (minimum 12 characters), multi-factor authentication, account lockout rate limiting[cite: 5, 6].
- **For R2:** Network segmentation with firewall rules denying guest access to the server, plus a regular patching schedule[cite: 5, 6].
- **For R3:** Encrypt files before transfer (AES-256-GCM) and use secure channels, with SHA-256 integrity checks[cite: 5, 6].

3 Encryption and Integrity Application

3.1 Design

The program `src/secure_records.py` is written in Python and uses the cryptography library[cite: 5, 6].

Encryption: AES-256-GCM authenticated encryption with a random 256-bit key stored securely outside the repository[cite: 5, 6].

Decryption: Validates the nonce and tag, failing safely if tampered with[cite: 5, 6].

Integrity: Computes and stores SHA-256 digests in a baseline file (`hashes.json`) to detect changes (`UNCHANGED` or `MODIFIED`)[cite: 5, 6].

Error Handling: Missing files, empty files, and invalid keys are caught and reported cleanly without throwing stack traces[cite: 5, 6].

3.2 Test evidence

All unit tests and automated CLI execution checks (encryption, decryption, hash verification, tamper detection, and error handling) pass successfully with exit codes 0, 1, or 2 as expected[cite: 5, 6].

4 Network Traffic Filtering

4.1 Laboratory setup

- **Records Server IP:** `192.168.100.10`[cite: 1, 5, 6]
- **Service Tested:** `tcp/443` (HTTPS)[cite: 1, 5, 6]

- **Staff Client (Permitted):** ‘192.168.10.50’ in network ‘192.168.10.0/24’[cite: 1, 5, 6]
- **Guest Client (Blocked):** ‘10.0.99.15’ in network ‘10.0.99.0/24’[cite: 1, 5, 6]
- **Firewall Tool:** ‘iptables’ via script ‘firewall/records_{server}firewall.sh’[cite : 1, 5, 6]

4.2 Firewall rules

Rules are managed via a dedicated ‘RECORDS_{FILTER}’ chain in ‘iptables’[cite : 1, 5, 6] :

1. *Accept established/related and loopback traffic*[cite : 5, 6]. 2. *Block guest network access to the server*(‘*—s GUEST_{NET} —j DROP*’)[cite : 1, 5, 6]. 3. *Permit staff network access to the designated service port*(‘*—s STAFF_{NET} —ptcp —dport 443 —j ACCEPT*’)[cite : 1, 5, 6]. 4. *Block all other inbound attempts to that service*[cite : 1, 5, 6].

4.3 Test procedure and results

Tested using netcat (‘nc -zv -w 5’)[cite: 1, 5, 6]:

- **T1 (Staff permitted):** ‘Connection to 192.168.100.10 443 port [tcp/https] succeeded’[cite: 1, 5, 6]
- **T2 (Guest blocked):** ‘nc: connect to 192.168.100.10 port 443 (tcp) timed out’[cite: 1, 5, 6]
- **T3 (Other external blocked):** ‘nc: connect to 192.168.100.10 port 443 (tcp) timed out’[cite: 1, 5, 6]

5 Conclusion

The assessment successfully identified and mitigated the core risks through a Python cryptographic toolkit and strict ‘iptables’ firewall configurations[cite: 1, 5, 6]. All tests behaved as expected, validating the security controls[cite: 1, 5, 6].

GitHub Repository URL: <https://github.com/rubimburagad85-hash/cryptography-network-security>[cite: 1, 5, 6]